

Plan de Proyecto SubastaLive

PLAN DE PROYECTO — v4

Plataforma Cloud Native de Subastas en Línea

Asignatura:	DSY1107 — Desarrollo Cloud Native I
Institución:	Duoc UC
Tipo de documento:	Plan de proyecto y hoja de ruta de entregas
Alcance:	Evaluaciones Parciales N°1, N°2 y N°3
Integrantes:	Juan Salas — Javier Aldea — Emerson Marchant
Docente:	Ignacio Cuturrufo
Sección:	I_004D
Fecha:	1 de septiembre de 2026

1. Introducción

Este documento presenta SubastaLive, una plataforma de subastas en línea desarrollada bajo un enfoque cloud native en el marco de la asignatura DSY1107.

El proyecto se construye en tres etapas sucesivas, cada una asociada a una evaluación parcial. Cada etapa parte del sistema funcional que dejó la anterior y le agrega una capa de capacidades. Esta característica —la incorporación aditiva, sin rehacer lo ya construido— es el eje conductor del plan y también el criterio con el que las pautas evalúan la calidad del diseño.

El documento describe el dominio elegido, las historias de usuario que motivan el sistema, los requisitos que de ellas se derivan, las decisiones de arquitectura y la hoja de ruta por etapas. Cada etapa recibe un capítulo con los requerimientos que la pauta exige, los microservicios y elementos que se implementarán para cumplirlos y una tabla de verificación. La Etapa 1 ya fue construida y desplegada (Entrega N°1); las secciones correspondientes reflejan su estado real. Las Etapas 2 y 3 todavía no comienzan y se describen como planificación.

2. Descripción del proyecto

2.1 Contexto y problemática

Una casa de remates que opera en línea enfrenta cargas de trabajo irregulares: una subasta puede permanecer con actividad mínima durante horas y concentrar gran parte de las pujas en sus últimos instantes. Ese comportamiento plantea, entre otras, cuatro necesidades que la arquitectura busca atender:

- **Orden en la resolución de pujas**, de modo que ante ofertas equivalentes se respete la secuencia de llegada.
- **Actualización oportuna del estado** para todos los participantes conectados.
- **Trazabilidad** del historial de pujas con fines de auditoría.
- **Desacoplamiento** de las tareas secundarias (avisos, comprobantes, cobros) respecto del flujo principal.

El plan aborda estas necesidades de forma progresiva a lo largo de las tres etapas, incorporando en cada una las tecnologías que la asignatura introduce.

2.2 Objetivo general

Diseñar, implementar y desplegar en la nube una plataforma de subastas en línea basada en microservicios que integre, de forma incremental, autenticación federada, mensajería asíncrona y procesamiento de eventos en streaming.

2.3 Objetivos específicos

#	Objetivo específico	Etapas
OE1	Construir un frontend SPA (Angular o React) con autenticación mediante IDaaS (OAuth 2.0 / OIDC)	1
OE2	Implementar microservicios Spring Boot con integración a base de datos cloud	1

OE3	Proteger los endpoints validando el JWT en el API Manager y en el backend	1
OE4	Incorporar mensajería asíncrona con RabbitMQ (colas, exchanges y DLQ)	2
OE5	Exponer administración programática de la topología de RabbitMQ	2
OE6	Incorporar Kafka para el procesamiento de eventos mediante tópicos	3
OE7	Habilitar productores y consumidores con consumer groups	3
OE8	Supervisar la plataforma de streaming mediante Kafka UI y métricas	3

2.4 Alcance

El sistema cubre el ciclo de una subasta: publicación del lote, apertura, recepción de pujas, cierre, adjudicación al mejor postor y notificación a los participantes. La definición precisa de las funcionalidades puede ajustarse conforme avance el desarrollo. Quedan fuera del alcance la pasarela de pago real —se simula el registro y la reserva del cobro— y la logística de despacho del lote adjudicado.

2.5 Actores y roles

Los actores y roles ya están **implementados de verdad**:

Rol	Descripción	Proveedor de identidad	Estado
Postor	Usuario registrado que participa en las subastas emitiendo pujas	Amazon Cognito	✅ Implementado y probado (login + logout reales, en local y en AWS)
Martillero	Usuario que publica lotes y administra las subastas	Microsoft Entra ID (Azure)	✅ Implementado y probado (login + logout reales, en local y en AWS)
Administrador	Usuario con permisos de gestión sobre la plataforma	Microsoft Entra ID (Azure)	✅ Implementado y probado (mismo tenant y app que Martillero, distinto app role)

El sistema utiliza dos proveedores de identidad diferenciados por tipo de usuario. Los postores son público general, con auto-registro masivo, para lo que Amazon Cognito resulta adecuado y económico a escala, además de integrarse de forma natural dentro de AWS. Los martilleros y administradores son usuarios de confianza con perfil operativo, gestionados en Microsoft Entra ID, que ofrece el control organizacional apropiado para ese grupo.

En ambos casos, el rol viaja dentro del token emitido por el proveedor correspondiente, de modo que el backend aplica la autorización a partir del claim de rol, con independencia de cuál IDaaS emitió el token. El conjunto de roles podrá ampliarse si el desarrollo lo requiere.

3. Historias de usuario y requisitos

Esta sección expresa las necesidades del sistema desde la perspectiva de quien lo usa, y deriva de cada historia los requisitos funcionales que la cubren. Las historias se agrupan por rol y se vinculan con la etapa en que se

implementan, de modo que la funcionalidad quede trazada de extremo a extremo: desde la necesidad del usuario hasta el requerimiento técnico y la entrega correspondiente.

3.1 Convenciones

Cada historia sigue el formato "Como <rol>, quiero <acción>, para <beneficio>". Los identificadores usan el prefijo **HU**. Los requisitos funcionales usan el prefijo **RF** y se numeran de forma continua. La columna "Etapa" indica cuándo la funcionalidad queda operativa.

3.2 Historias del Postor

HU-01 — Iniciar sesión de forma segura. Como postor, quiero iniciar sesión con mis credenciales a través de Amazon Cognito, para acceder a la plataforma sin que el sistema almacene mi contraseña.

Req.	Descripción	Etapa
RF-01	El sistema debe permitir el inicio de sesión del postor mediante Amazon Cognito con OAuth 2.0 / OIDC y flujo Authorization Code con PKCE	1
RF-02	El sistema debe obtener un token válido y adjuntarlo automáticamente en las llamadas al backend	1
RF-03	El sistema no debe almacenar credenciales; la identidad es responsabilidad del IDaaS	1
RF-31	El sistema debe permitir el auto-registro de nuevos postores a través de Cognito	1

HU-02 — Consultar subastas y lotes. Como postor, quiero ver las subastas disponibles y el detalle de cada lote, para decidir en cuáles participar.

Req.	Descripción	Etapa
RF-04	El sistema debe listar las subastas disponibles con su estado actual	1
RF-05	El sistema debe mostrar el detalle de un lote (descripción, precio base, imagen)	1

HU-03 — Emitir una puja. Como postor, quiero ofertar por un lote en una subasta abierta, para competir por adjudicármelo.

Req.	Descripción	Etapa
RF-06	El sistema debe aceptar pujas solo sobre subastas en estado abierto	1
RF-07	El sistema debe validar que el monto supere el precio actual más el incremento mínimo	1
RF-08	El sistema debe registrar la puja de forma persistente	1
RF-09	El sistema debe procesar las pujas de una misma subasta respetando su orden de llegada	3

HU-04 — Ver el precio actualizado en vivo. Como postor, quiero ver el precio y el estado de la subasta actualizarse sin recargar, para reaccionar a tiempo durante la puja.

Req.	Descripción	Etapas
RF-10	El sistema debe reflejar el estado y precio vigente de la subasta de forma oportuna	3
RF-11	El sistema debe notificar al postor cuando su oferta ha sido superada	3

HU-05 — Ser notificado del resultado. Como postor, quiero recibir un aviso cuando una subasta en la que participé se cierre y adjudique, para conocer el resultado.

Req.	Descripción	Etapas
RF-12	El sistema debe notificar de forma asíncrona el cierre y resultado de la subasta	2
RF-13	El envío de notificaciones no debe bloquear el cierre de la subasta	2

HU-06 — Consultar mi historial. Como postor, quiero consultar el historial de mis pujas y adjudicaciones, para llevar registro de mi participación.

Req.	Descripción	Etapas
RF-14	El sistema debe mantener un perfil de dominio del usuario con su actividad	1
RF-15	El sistema debe permitir consultar el historial de participación del postor	1

3.3 Historias del Martillero

HU-14 — Iniciar sesión como usuario operativo. Como martillero o administrador, quiero iniciar sesión a través de Microsoft Entra ID, para acceder a las funciones operativas con el control de una identidad organizacional.

Req.	Descripción	Etapas
RF-32	El sistema debe permitir el inicio de sesión de martilleros y administradores mediante Microsoft Entra ID con OAuth 2.0 / OIDC y flujo Authorization Code con PKCE	1
RF-33	El API Gateway debe aceptar y validar tokens de ambos proveedores de identidad (Cognito y Entra ID)	1
RF-34	El sistema debe resolver el rol del usuario a partir del token, con independencia del proveedor emisor	1

HU-07 — Publicar un lote y programar su subasta. Como martillero, quiero publicar un lote y programar la apertura y el cierre de su subasta, para ponerlo a disposición de los postores.

Req.	Descripción	Etapas
RF-16	El sistema debe permitir crear un lote con sus datos y precio base	1

RF-17	El sistema debe permitir programar la apertura y cierre de una subasta	1
RF-18	El sistema debe gestionar las transiciones de estado de la subasta	1

HU-08 — Que la subasta se cierre y adjudique automáticamente. Como martillero, quiero que al vencer el plazo la subasta se cierre y se determine el mejor postor, para no tener que resolver la adjudicación manualmente.

Req.	Descripción	Etapas
RF-19	El sistema debe determinar el mejor postor al cierre de la subasta	3
RF-20	El sistema debe generar de forma asíncrona el comprobante de adjudicación	2
RF-21	El sistema debe registrar de forma asíncrona la reserva del cobro al adjudicatario	2

HU-09 — Consultar analítica de mis subastas. Como martillero, quiero ver métricas en vivo de mis subastas (pujas por minuto, postores activos), para seguir su desempeño.

Req.	Descripción	Etapas
RF-22	El sistema debe calcular métricas de actividad a partir del flujo de pujas	3
RF-23	El cálculo de analítica debe operar de forma independiente del procesamiento de adjudicación	3

3.4 Historias del Administrador

HU-10 — Administrar la topología de mensajería. Como administrador, quiero crear y eliminar colas, exchanges y bindings de RabbitMQ, para gestionar la infraestructura de tareas asíncronas.

Req.	Descripción	Etapas
RF-24	El sistema debe exponer operaciones para crear y eliminar colas, exchanges y bindings	2
RF-25	Las operaciones de administración deben validar sus parámetros de entrada	2

HU-11 — Administrar los tópicos de streaming. Como administrador, quiero crear, modificar y eliminar tópicos y configuraciones de Kafka, para gestionar la plataforma de eventos.

Req.	Descripción	Etapas
RF-26	El sistema debe exponer operaciones para crear, modificar y eliminar tópicos y sus configuraciones	3
RF-27	El sistema debe permitir inspeccionar tópicos y grupos de consumidores	3

HU-12 — Supervisar el estado del streaming. Como administrador, quiero observar métricas de los tópicos y consumidores (offsets, lag, distribución), para anticipar problemas operacionales.

Req.	Descripción	Etapas
------	-------------	--------

RF-28	El sistema debe permitir supervisar offsets, lag y distribución del consumo mediante Kafka UI	3
-------	---	---

HU-13 — Garantizar el control de acceso. Como administrador, quiero que cada operación quede protegida según el rol del usuario, para que solo quien corresponde pueda ejecutarla.

Req.	Descripción	Etapas
RF-29	El sistema debe validar el JWT en el borde y en cada microservicio (defensa en profundidad)	1
RF-30	El sistema debe resolver la autorización fina por rol dentro de los microservicios	1

3.5 Requisitos no funcionales

Req.	Descripción	Etapas
RNF-01	Cada microservicio debe poder escalar de forma independiente	1
RNF-02	El sistema debe conservar el historial de pujas para auditoría posterior	3
RNF-03	Los consumidores de eventos deben ser idempotentes ante reprocesamiento	3
RNF-04	Los mensajes no procesables deben derivarse a una cola de mensajes muertos (DLQ) y registrarse	2
RNF-05	El sistema debe controlar el uso de conexiones a la base de datos por contenedor	1
RNF-06	Cada microservicio debe ser dueño de sus datos, sin acceso directo de otros a su esquema	1

3.6 Trazabilidad historias → requisitos → etapas

Historia	Requisitos funcionales	Etapas principales
HU-01 Iniciar sesión postor (Cognito)	RF-01, RF-02, RF-03, RF-31	1
HU-02 Consultar subastas	RF-04, RF-05	1
HU-03 Emitir puja	RF-06, RF-07, RF-08, RF-09	1 / 3
HU-04 Precio en vivo	RF-10, RF-11	3
HU-05 Notificación de resultado	RF-12, RF-13	2
HU-06 Historial	RF-14, RF-15	1
HU-07 Publicar lote	RF-16, RF-17, RF-18	1
HU-08 Cierre y adjudicación	RF-19, RF-20, RF-21	2 / 3
HU-09 Analítica	RF-22, RF-23	3
HU-10 Administrar RabbitMQ	RF-24, RF-25	2

HU-11 Administrar Kafka	RF-26, RF-27	3
HU-12 Supervisar streaming	RF-28	3
HU-13 Control de acceso	RF-29, RF-30	1
HU-14 Iniciar sesión operativo (Entra ID)	RF-32, RF-33, RF-34	1

3.7 Estado real de las historias de la Etapa 1

Historia	Estado
HU-01 Iniciar sesión postor (Cognito)	✓ Implementada
HU-02 Consultar subastas	✓ Implementada (<code>ms-catalogo</code> , con datos de ejemplo — persistencia real pendiente)
HU-03 Emitir puja (RF-06, RF-07, RF-08)	✓ Implementada en <code>ms-pujas</code> , con persistencia real en RDS
HU-06 Historial	✦ Contrato implementado en <code>ms-usuarios</code> , sin persistencia real ni validación JWT todavía
HU-07 Publicar lote	✦ Contrato implementado en <code>ms-catalogo</code> (stub), sin persistencia real todavía
HU-13 Control de acceso (RF-29, RF-30)	✓ Implementado en <code>ms-pujas</code> (defensa en profundidad real). Pendiente en <code>ms-usuarios</code> / <code>ms-catalogo</code>
HU-14 Iniciar sesión operativo (Entra ID)	✓ Implementada

4. Estrategia de desarrollo incremental

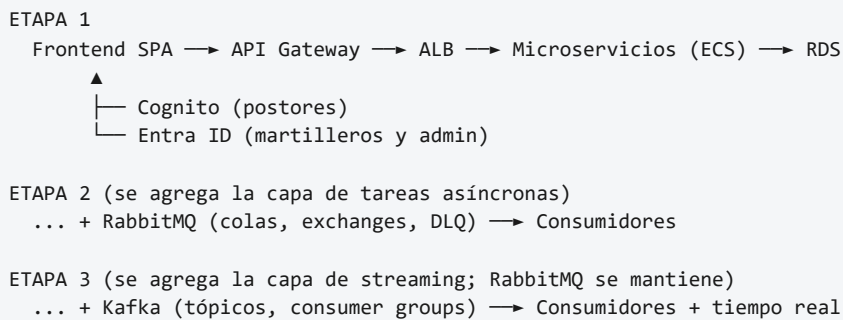
4.1 Enfoque por etapas

Cada etapa constituye una entrega funcional: al término de cualquiera de ellas el sistema queda desplegado y operativo.

Etapas	Evaluación	Foco	Capacidad que incorpora
1	Parcial N°1	Arquitectura base e identidad	Autenticación federada, exposición segura, persistencia
2	Parcial N°2	Mensajería asíncrona	Desacoplamiento de tareas secundarias
3	Parcial N°3	Streaming de eventos	Procesamiento de eventos, analítica y tiempo real

La secuencia es dependiente: la Etapa 2 requiere los microservicios y la identidad de la Etapa 1, y la Etapa 3 requiere un flujo de negocio funcionando para tener eventos que publicar.

4.2 Evolución prevista de la arquitectura



El diagrama es indicativo. La composición final de cada capa se afina durante el desarrollo.

4.3 Criterio de no regresión

Al cierre de cada etapa se verifica que las capacidades de las etapas anteriores sigan operando. Este criterio responde a lo que plantean las pautas: la Etapa 2 pide demostrar que la mensajería funciona sin afectar la lógica existente, y la Etapa 3 pide demostrar que Kafka complementa la arquitectura sin afectar las integraciones previas con RabbitMQ.

5. Decisiones de arquitectura

Esta sección consolida las decisiones técnicas que gobiernan todo el proyecto, de modo que las etapas siguientes se apoyen en criterios ya establecidos.

5.1 Cómputo y despliegue — confirmado, con un ajuste real

Se confirma **Amazon ECS con Fargate** para los microservicios de aplicación, cada uno como un servicio independiente que puede escalar por separado (RNF-01). Un Application Load Balancer (ALB) distribuye el tráfico entre las réplicas de cada servicio. El API Gateway enruta hacia el ALB, no directamente a las tareas.

La infraestructura de mensajería (RabbitMQ y Kafka, Etapas 2 y 3) se ejecutará con Docker Compose sobre EC2, tal como exige la pauta, que requiere demostrar su configuración interna. El resultado es una arquitectura deliberadamente mixta: aplicación en ECS/Fargate, mensajería en EC2. En un entorno productivo la mensajería podría delegarse a servicios gestionados como Amazon MQ y Amazon MSK; el uso de Docker Compose responde al objetivo académico de demostrar la operación de estas plataformas.

Un ajuste real respecto a lo previsto:

El frontend no se desplegó como sitio estático en S3, como se había planeado inicialmente. El laboratorio de AWS Academy usado en este proyecto no otorga el permiso `cloudfront:CreateOriginAccessControl` (`AccessDenied`) — ni siquiera con un origen ALB en vez de S3, CloudFront queda bloqueado por completo (`cloudfront:CreateDistribution` también da `AccessDenied`). Como ECR/ECS/ALB sí funcionan sin problema en esta cuenta, el frontend se despliega **igual que un microservicio más**: un contenedor Nginx sirviendo el build de Vite, detrás de su propio ALB. Ver `despliegue-aws.md`, nota de la sección 10, para el detalle completo.

Otro ajuste real, de costo/diseño:

Los tres microservicios backend comparten un solo ALB (`subastalive-alb`), con reglas de listener por path (`/subastas*` + `/lotes*` → `ms-catalogo` , `/usuarios*` → `ms-usuarios` , todo lo demás → `ms-pujas` por defecto) — en vez de un ALB por microservicio. Ninguno de los tres es alcanzado directamente por el navegador (siempre pasan por el API Gateway o por llamadas internas servicio-a-servicio), así que no había motivo para pagar por 3 ALB corriendo todo el tiempo. El frontend sí mantiene su **propio** ALB, porque tiene un consumidor distinto (el navegador) y necesita su propio certificado HTTPS.

5.2 Persistencia — confirmado

El estado transaccional se almacena en **PostgreSQL sobre Amazon RDS**. Se elige un motor relacional porque los datos del dominio son fuertemente transaccionales y con dinero de por medio, lo que exige garantías ACID, y porque las entidades están naturalmente relacionadas.

Se utiliza una instancia RDS compartida con **un esquema separado por microservicio** (RNF-06). Cada servicio es dueño de su esquema y ningún otro accede a él directamente.

```
Amazon RDS - PostgreSQL
├─ schema_usuarios → ms-usuarios
├─ schema_catalogo → ms-catalogo
└─ schema_pujas   → ms-pujas
```

Para evitar el agotamiento de conexiones cuando Fargate escale horizontalmente, se configura el pool de conexiones (HikariCP) con un límite máximo reducido por contenedor (RNF-05). Como evolución hacia un entorno de alta demanda se contempla la incorporación de Amazon RDS Proxy; no forma parte del alcance de las entregas.

Confirmado en la práctica con Flyway en `ms-pujas` : migra su esquema (`schema_pujas`) automáticamente al arrancar, sin ningún paso manual contra RDS — ni la primera vez ni en despliegues posteriores. `ms-catalogo` y `ms-usuarios` (stubs) todavía no persisten en RDS de verdad.

Aclaración de decisión: como el proyecto terminó con microservicios en más de un stack (`ms-pujas` en Spring Boot/Java, `ms-catalogo` / `ms-usuarios` con intención de Node), la migración automática de esquema **no tiene que ser Flyway específicamente** — Flyway es una herramienta del ecosistema Java. Un microservicio en Node puede lograr el mismo mecanismo (migraciones versionadas, sin acceso manual a RDS) con una herramienta nativa como `node-pg-migrate` . El requisito real es el mecanismo, no la herramienta puntual.

5.3 Mensajería: dos plataformas complementarias

El sistema utiliza deliberadamente RabbitMQ y Kafka porque resuelven problemas distintos:

Tecnología	Responsabilidad	Ejemplo
PostgreSQL	Estado transaccional actual	Mejor puja vigente
RabbitMQ	Distribución de trabajos (comandos)	Generar comprobante de pago
Kafka	Distribución de eventos de dominio	PUJA_REALIZADA

RabbitMQ responde a "¿quién debe ejecutar este trabajo?", Kafka a "¿qué ocurrió y quién necesita enterarse?", y PostgreSQL a "¿cuál es el estado actual?". Esta separación evita forzar una sola tecnología a resolver problemas diferentes. No aplica todavía (Etapa 2, no iniciada).

5.4 Consistencia entre base de datos y eventos

Registrar una puja implica escribir en PostgreSQL y publicar en Kafka. Como son dos sistemas distintos, esta doble escritura no es atómica. La estrategia adoptada, priorizando simplicidad y robustez, es:

1. **ms-pujas escribe la puja en RDS y luego publica el evento en Kafka.** Se acepta explícitamente que la operación no es atómica entre ambos sistemas.
2. **Los consumidores de eventos son idempotentes (RNF-03):** al recibir un evento verifican su estado contra la base de datos y descartan de forma segura los eventos ya procesados o huérfanos.

Esta combinación mitiga el riesgo de inconsistencia con un mecanismo simple y verificable. Como evolución hacia consistencia fuerte se contempla el patrón *Transactional Outbox*; no forma parte del alcance de las entregas. No aplica todavía (Etapa 3, no iniciada).

5.5 Catálogo de microservicios por etapa — estado real de la Etapa 1

La tabla siguiente consolida qué microservicios existen en cada etapa (planificación completa). Cada etapa **agrega** servicios sin retirar los anteriores.

Microservicio	Etapa 1	Etapa 2	Etapa 3	Responsabilidad
ms-usuarios	•	•	•	Perfil de dominio del usuario, vinculado al sub del token
ms-catalogo	•	•	•	Lotes, subastas, estados, aperturas y cierres
ms-pujas	•	•	•	Recepción y validación de pujas; productor Kafka en Etapa 3
ms-notificaciones		•	•	Consumidor RabbitMQ: avisos a los participantes
ms-documentos		•	•	Consumidor RabbitMQ: comprobante de adjudicación
ms-pagos		•	•	Consumidor RabbitMQ: reserva del cobro
ms-admin-rabbit		•	•	Administración de la topología de RabbitMQ
ms-adjudicacion			•	Consumidor Kafka: determina el mejor postor y cierra
ms-analitica			•	Consumidor Kafka: métricas en vivo
ms-admin-kafka			•	Administración de tópicos y configuraciones de Kafka

Total acumulado: 3 microservicios en la Etapa 1, 7 en la Etapa 2, 10 en la Etapa 3.

- **El rol de productor se traslada entre etapas.** En la Etapa 2, el evento de cierre que dispara las tareas asíncronas lo publica ms-catalogo al cerrar la subasta. En la Etapa 3, cuando aparece ms-adjudicacion , ese rol se traslada a este último. No es un microservicio nuevo, es un rol que cambia de dueño.
- **No existe un ms-persistencia dedicado.** Conforme a la decisión de la sección 5.4, la escritura en RDS ocurre en ms-pujas (el origen), por lo que los consumidores de Kafka son adjudicación y analítica, sin un

consumidor exclusivo de persistencia.

- **Los administradores son servicios livianos.** `ms-admin-rabbit` y `ms-admin-kafka` encapsulan operaciones de infraestructura y no participan del flujo de negocio.

Estado real de la Etapa 1:

Microservicio	Previsto	Estado real
<code>ms-usuarios</code>	Spring Boot (planeado originalmente, libre)	Stub liviano en Node/Express , desplegado en AWS. Sin validación JWT real todavía
<code>ms-catalogo</code>	—	Stub liviano en Node/Express , desplegado en AWS. Sin persistencia real todavía
<code>ms-pujas</code>	—	Implementado de verdad en Java/Spring Boot. Persistencia real (RDS + Flyway), validación JWT real multi-issuer, probado en Docker Compose local y desplegado en ECS/Fargate

El stack de cada microservicio quedó libre — el equipo terminó dividido entre Java (`ms-pujas`) y Node (`ms-catalogo` / `ms-usuarios`), sin que eso afecte el contrato JSON compartido entre ellos.

5.6 Identidad con dos proveedores — confirmado e implementado

El sistema autentica a sus usuarios contra dos proveedores de identidad distintos, elegidos según el perfil del usuario:

Grupo de usuarios	Proveedor	Motivo
Postores (clientes)	Amazon Cognito	Público general con auto-registro masivo; económico a escala; integrado en AWS
Martilleros y administradores	Microsoft Entra ID	Usuarios operativos de confianza; control organizacional

Ambos proveedores emiten tokens siguiendo OAuth 2.0 / OIDC con flujo Authorization Code con PKCE, por lo que el frontend los trata de manera uniforme: cambia el proveedor al que redirige el login, no la mecánica del flujo.

Frontend único con doble entrada de login. Se opta por un único frontend SPA en lugar de aplicaciones separadas por rol. La aplicación ofrece una zona pública sin autenticación, un ingreso "como postor" que redirige a Cognito y un ingreso "como martillero/administrador" que redirige a Entra ID, y renderiza las vistas según el rol que trae el token.

La decisión se implementó exactamente como se diseñó, incluyendo:

- Ambos con Authorization Code + PKCE real (verificado con `oidc-client-ts` en el frontend).
- El API Gateway valida JWT con un autorizador nativo contra Cognito, funcionando en producción.

Matiz real encontrado: un autorizador JWT nativo de API Gateway valida contra **un solo issuer**. El diseño original ya anticipaba que esto se resolvería "mediante autorizadores diferenciados por emisor o un autorizador que valide contra ambos, sin alterar el diseño" — en la práctica, con Cognito ya en producción, se usó el autorizador nativo de un

solo issuer para postores; incorporar Entra ID al mismo Gateway queda pendiente para cuando el flujo de martillero/administrador necesite llamar a los microservicios vía Gateway.

Aclaración de terminología, surgida al revisar la rúbrica de la asignatura: la pauta de evaluación (plantilla genérica de Duoc, no específica de este proyecto) usa el término "BFF" para referirse a que el **backend** valide el JWT igual que el API Manager — no al patrón arquitectónico real de "Backend for Frontend" (un servicio que agrega datos de varios microservicios para un frontend específico). SubastaLive no tiene ni necesita un BFF real: no hay agregación de datos entre microservicios de cara al frontend, y agregar uno introduciría una tercera capa de validación JWT redundante sin aportar seguridad adicional — los microservicios ya validan el JWT por su cuenta (defensa en profundidad, RF-29), independiente del API Gateway. El indicador de la pauta se cumple con la validación ya implementada en `ms-pujas`.

6. Etapa 1 — Arquitectura base, identidad y exposición segura

6.1 Requerimientos de la etapa

Según la pauta de la Evaluación Parcial N°1:

Encargo (código fuente):

#	Requerimiento
R1.1	Backend compuesto por varios microservicios en Java con Spring Boot
R1.2	Frontend SPA (Angular o React) completo, modular, sin errores de compilación y con vistas funcionales
R1.3	Código del backend que compile, siga buenas prácticas y responda a pruebas básicas
R1.4	Integración con base de datos cloud mediante entidades, repositorios y propiedades de conexión
R1.5	Filtros en el backend que validen el JWT recibido desde el IDaaS
R1.6	Flujo de login con IDaaS en el frontend, utilizando el JWT en las llamadas al backend
R1.7	Archivos <code>.gitignore</code> que suban a cada repositorio solo lo que corresponde a su tecnología
R1.8	Entrega mediante enlaces a repositorios GitHub

Presentación (demostración):

#	Requerimiento
R1.9	Instancia de API Manager creada y en funcionamiento en la plataforma cloud
R1.10	Configuración del API Manager que permita llamar a los endpoints del backend
R1.11	Frontend consumiendo los endpoints a través del API Manager
R1.12	API Manager validando JWT: rechaza peticiones inválidas y acepta las correctas

R1.13	Tenant creado en el IDaaS con usuarios registrados
R1.14	Frontend usando OAuth 2.0 / OpenID Connect para iniciar sesión y obtener un JWT válido
R1.15	Backend y frontend desplegados, activos e integrados en la nube

La pauta pondera además, en la presentación, la creación y configuración del tenant, el registro de la aplicación dentro del tenant, el flujo de registro e inicio de sesión, el uso del flujo Authorization Code con PKCE, la validación de JWT en todas las rutas y la evidencia del funcionamiento de cada ruta.

6.2 Microservicios de la etapa

Microservicio	Responsabilidad	Historias que cubre
ms-usuarios	Perfil de dominio del usuario, vinculado al sub del token; historial de participación	HU-06
ms-catalogo	Lotes, subastas, estados, programación de aperturas y cierres	HU-02, HU-07
ms-pujas	Recepción y validación de pujas (subasta abierta, monto sobre el mínimo)	HU-03

Cada microservicio identifica al usuario por el sub presente en el token, sin llamar a ms-usuarios en cada operación; ms-usuarios es dueño del perfil de dominio, no un intermediario de identidad.

6.3 Elementos a implementar

Frontend (SPA único). Una única aplicación SPA con una zona pública sin autenticación y dos entradas de login diferenciadas: "ingresar como postor" (redirige a Cognito) e "ingresar como martillero/administrador" (redirige a Entra ID), renderizando las vistas según el rol del token, conforme a la sección 5.6. Se construyó en **React + Vite**, con **oidc-client-ts** para la integración OIDC. En ambas entradas el login usa OAuth 2.0 / OIDC mediante Authorization Code con PKCE, y el token se adjunta automáticamente en las llamadas al backend mediante un interceptor. Se usan rutas protegidas según el rol.

Backend de microservicios. Los tres microservicios de la etapa integran su esquema en RDS mediante entidades, repositorios y propiedades de conexión externalizadas, con el pool de HikariCP acotado por contenedor. Cada microservicio incorpora un filtro que valide el JWT antes de autorizar las peticiones y resuelve la autorización por rol de los casos que lo requieran.

Identidad (dos proveedores). Conforme a la sección 5.6, se configuraron dos proveedores. En Amazon Cognito se creó un user pool para los postores, con auto-registro habilitado, la aplicación cliente registrada, las URIs de redirección y los roles necesarios. En Microsoft Entra ID se creó un tenant para martilleros y administradores, con la aplicación registrada, las URIs de redirección, los roles y scopes, y usuarios de prueba por rol. Ambos se configuran para emitir tokens mediante Authorization Code con PKCE.

API Manager. Se creó una HTTP API en Amazon API Gateway como único punto de entrada. Contempla las rutas hacia los microservicios (a través del ALB), la configuración de CORS con el origen del frontend, y la validación de JWT.

Validación en el backend. Además del borde, cada microservicio revalida el token de forma independiente, estableciendo defensa en profundidad (RF-29).

Despliegue. Los microservicios se despliegan containerizados en ECS/Fargate tras el ALB compartido. El frontend se despliega igual que un microservicio más (ver sección 5.1). Se configuraron los `.gitignore` por tecnología y la entrega es mediante enlaces a GitHub.

6.4 Verificación de requerimientos — estado real, Entrega N°1

Req.	Elemento que lo cubre	Estado
R1.1	Tres microservicios separados por dominio	✓ <code>ms-pujas</code> en Spring Boot; los otros dos en Node/Express (stubs)
R1.2	Aplicación SPA con vistas funcionales	✓ React + Vite, diseño propio, en AWS
R1.3	Build reproducible y pruebas básicas	✓ Tests Mockito en <code>ms-pujas</code> ; build Docker reproducible en los 4 componentes
R1.4	Entidades, repositorios y esquema en RDS	✓ <code>ms-pujas</code> (JPA + Flyway). ♦ stubs sin persistencia real
R1.5	Filtro de validación de JWT por microservicio	✓ <code>ms-pujas</code> . ✗ <code>ms-usuarios</code> decodifica pero no verifica firma/issuer
R1.6	Login OIDC y adjunción del token	✓ Cognito y Entra ID, con logout real incluido
R1.7	<code>.gitignore</code> por tecnología	✓ Los 4 componentes + <code>infra-terraform/</code>
R1.8	Repositorios en GitHub	✓ Monorepo único, con ramas de feature
R1.9	Instancia de API Gateway desplegada	✓ HTTP API real, con auto-deploy
R1.10	Rutas del API Gateway vía ALB	✓ Un solo <code>/{{proxy+}}</code> sirve a los 3, porque el ALB compartido enruta por path
R1.11	Frontend consumiendo vía API Gateway	♦ Probado con Postman/curl; frontend desplegado sigue en modo simulado
R1.12	Validación JWT multi-issuer en el Gateway	✓ Cognito en producción (401/200 reales). Entra ID pendiente
R1.13	User pool y tenant con usuarios	✓ Ambos con usuarios de prueba por rol
R1.14	Authorization Code con PKCE	✓ Confirmado en ambos proveedores
R1.15	Backend y frontend desplegados e integrados	✓ Los 4 componentes en ECS/Fargate, con CI/CD real

Resumen: 12 de 15 requisitos completamente cumplidos, 3 con cumplimiento parcial — ninguno de los pendientes requiere cambios de arquitectura, son continuaciones directas de lo ya construido.

7. Etapa 2 — Mensajería asíncrona con RabbitMQ

No iniciada.

7.1 Requerimientos de la etapa

Según la pauta de la Evaluación Parcial N°2:

Encargo:

#	Requerimiento
R2.1	Productores y consumidores en los microservicios, conectados a colas RabbitMQ
R2.2	Manejo de mensajes no entregados enviándolos a DLQ y registrándolos en logs
R2.3	Microservicio administrador que permita crear y eliminar colas, exchanges y bindings
R2.4	Nombres de colas, exchanges y bindings definidos de forma centralizada (<code>application.yml</code> o <code>@Configuration</code>)
R2.5	Beans de configuración de <code>Queue</code> , <code>Exchange</code> y <code>Binding</code> por cada caso de uso
R2.6	Separación entre la lógica de negocio y la configuración de mensajería
R2.7	Consumidores implementados con <code>@RabbitListener</code> , agrupados por dominio funcional
R2.8	Confirmación de mensajes con ACK y manejo explícito de errores
R2.9	Endpoints REST bien definidos en el microservicio administrador
R2.10	Lógica de administración encapsulada en un servicio dedicado
R2.11	Validación de parámetros de entrada para evitar configuraciones inválidas

Presentación:

#	Requerimiento
R2.12	Dos nodos RabbitMQ configurados e integrados en un clúster
R2.13	Tres colas de mensajes con sus respectivas DLQ funcionando
R2.14	Exchanges de tipo <code>direct</code> y <code>topic</code> correctamente configurados
R2.15	Ejecución en la nube de <code>docker-compose</code> levantando los servicios de RabbitMQ
R2.16	Se mantienen los elementos demostrados en la Etapa 1

7.2 Punto de partida e incremento previsto

Al término de la Etapa 1, las tareas secundarias de una adjudicación (avisos, comprobantes, registro de cobro) se resuelven de forma sincrónica dentro del flujo principal. El incremento de esta etapa consiste en extraer esas tareas del flujo sincrónico y tratarlas como mensajes que se procesan de forma independiente mediante RabbitMQ (HU-05, HU-08).

7.3 Microservicios de la etapa

Se agregan cuatro microservicios a los tres existentes:

Microservicio	Rol	Historias que cubre
ms-notificaciones	Consumidor RabbitMQ	HU-05
ms-documentos	Consumidor RabbitMQ	HU-08 (comprobante)
ms-pagos	Consumidor RabbitMQ	HU-08 (cobro)
ms-admin-rabbit	Administrador de topología	HU-10

En esta etapa, `ms-catalogo` asume el rol de productor: al cerrar una subasta publica el evento que dispara las tres tareas asíncronas, y responde sin esperar a que concluyan (RF-13).

7.4 Elementos a implementar

Productores y consumidores. `ms-catalogo` publica el evento de cierre; los tres consumidores se implementan con `@RabbitListener`, agrupados por dominio.

Topología de mensajería. Se configuran un exchange `topic` (para lo que requiere enrutamiento por patrón, como notificaciones y pagos) y un exchange `direct` (para el destino determinístico de documentos), tres colas con sus respectivas DLQ, y los nombres se centralizan en configuración.

Configuración mediante beans. Se declaran beans de `Queue`, `Exchange` y `Binding` por caso de uso, en clases separadas de la lógica de negocio.

Manejo de mensajes no entregados. Los mensajes no procesables se derivan a la DLQ correspondiente y se registran en logs (RNF-04). El manejo de errores distingue de forma binaria lo recuperable (reintento acotado) de lo no recuperable (directo a DLQ).

Microservicio administrador. `ms-admin-rabbit` expone endpoints REST para crear y eliminar colas, exchanges y bindings, con la lógica encapsulada en un servicio dedicado y validación de entrada mediante DTO.

Clúster y despliegue. Dos nodos RabbitMQ en clúster mediante un `docker-compose` funcional sobre EC2.

7.5 Continuidad de la Etapa 1

Se verificará que la autenticación, las rutas y validación del API Gateway, la validación de JWT en el backend y la persistencia sigan operando sin cambios. El único cambio sobre el código existente es que `ms-catalogo`, al cerrar una subasta, publica un mensaje en lugar de ejecutar las tareas de forma sincrónica; la lógica de negocio de la adjudicación permanece intacta.

7.6 Verificación de requerimientos

Requerimiento	Elemento que lo cubre
R2.1	ms-catalogo productor; tres consumidores conectados a colas
R2.2	Tres DLQ con registro en logs
R2.3	ms-admin-rabbit con operaciones de topología
R2.4	Nombres centralizados en configuración
R2.5	Beans de Queue, Exchange y Binding por caso de uso
R2.6	Configuración separada de la lógica de negocio
R2.7	Consumidores @RabbitListener por dominio
R2.8	ACK manual y manejo binario de errores
R2.9	Endpoints REST del administrador
R2.10	Lógica encapsulada en servicio dedicado
R2.11	Validación de parámetros con DTO
R2.12	Clúster de dos nodos
R2.13	Tres colas con DLQ
R2.14	Exchanges direct y topic
R2.15	docker-compose de RabbitMQ sobre EC2
R2.16	Continuidad de la Etapa 1

8. Etapa 3 — Streaming de eventos con Kafka

No iniciada.

8.1 Requerimientos de la etapa

Según la pauta de la Evaluación Parcial N°3:

Encargo:

#	Requerimiento
R3.1	Configuración completa del cliente Kafka con conexión a los brokers
R3.2	Productores y consumidores conectados a los tópicos, con estructuras tipadas (DTO)

R3.3	Microservicio administrador de Kafka: creación, modificación y eliminación de tópicos y configuraciones
R3.4	Productores Kafka enviando eventos con serializers adecuados
R3.5	Consumidores con <code>@KafkaListener</code> , con la lógica de procesamiento separada de la mensajería
R3.6	Manejo de errores: logging, validación de payloads o control de reintentos
R3.7	Parámetros de administración de tópicos: retención, cleanup policy, particiones y factor de replicación
R3.8	Consumer groups configurados para balancear carga o asegurar procesamiento resiliente
R3.9	<code>docker-compose</code> funcional para levantar el clúster de RabbitMQ
R3.10	<code>docker-compose</code> funcional para la arquitectura Kafka con Zookeeper y brokers
R3.11	<code>docker-compose</code> funcional para Kafka UI

Presentación:

#	Requerimiento
R3.12	Explicación de la arquitectura Kafka: rol de los tres nodos Zookeeper y los tres brokers
R3.13	Justificación de dos tópicos con tres particiones y tres réplicas cada uno
R3.14	Demostración de Kafka UI: administración de brokers, tópicos, particiones y mensajes
R3.15	Explicación del flujo completo de producción y consumo de eventos
R3.16	Descripción del diseño de los eventos y de por qué contienen los datos necesarios
R3.17	Demostración de la respuesta del sistema a eventos en tiempo real o cuasi real
R3.18	Explicación de administración de Kafka: particiones, réplicas, retention y consumer groups
R3.19	Supervisión mediante métricas: offsets, lag y distribución del consumo
R3.20	Descripción de los casos de uso que requieren streaming
R3.21	Se mantienen los elementos demostrados en las Etapas 1 y 2

8.2 Punto de partida e incremento previsto

Al término de la Etapa 2, el sistema tiene desacopladas sus tareas secundarias, pero el flujo de pujas se procesa sin las capacidades de una plataforma de streaming: orden por subasta, múltiples consumidores del mismo evento y retención para auditoría. El incremento consiste en incorporar Kafka para tratar el flujo de pujas, manteniendo RabbitMQ para las tareas asíncronas existentes (HU-04, HU-09, RNF-02).

8.3 Microservicios de la etapa

Se agregan tres microservicios a los siete existentes:

Microservicio	Rol	Historias que cubre
ms-adjudicacion	Consumidor Kafka: determina el mejor postor y cierra	HU-08 (adjudicación)
ms-analitica	Consumidor Kafka: métricas en vivo	HU-09
ms-admin-kafka	Administrador de tópicos y configuraciones	HU-11, HU-12

ms-pujas asume el rol de productor Kafka: publica el evento PUJA_REALIZADA tras validar cada puja. El rol de publicar el evento de cierre que en la Etapa 2 tenía ms-catalogo se traslada a ms-adjudicacion, que detecta la condición de cierre desde el flujo de eventos.

8.4 Elementos a implementar

Configuración del cliente Kafka. Conexión a los brokers levantados en Docker Compose, con bootstrap servers, serializers, consumer groups y las propiedades para publicar y consumir. En el productor se habilita idempotencia y acks=all.

Diseño de tópicos. Dos tópicos, cada uno con tres particiones y tres réplicas. El tópico de pujas usa auctionId como clave del mensaje, de modo que las pujas de una misma subasta caigan en la misma partición y conserven su orden (RF-09), sin exigir un orden global. Se ajustan retención, cleanup policy, particiones y factor de replicación según cada flujo. El riesgo de saturación de una partición (*hot partitioning*) se reconoce como consideración; su mitigación mediante rate limiting o micro-batching queda como evolución, fuera del alcance de la entrega.

Diseño de eventos. Los eventos se modelan como DTO tipados que contienen los datos necesarios para resolver los casos de uso: identificador de la subasta, de la puja y del usuario, monto, y fecha y hora. Se serializan en JSON.

Productores y consumidores. ms-pujas produce; los consumidores se implementan con @KafkaListener manteniendo la lógica de dominio separada de la mensajería.

Consumer groups. Grupos independientes para adjudicación y analítica, de modo que ambos procesen el mismo evento sin interferirse (RF-23). Dentro de un grupo, las instancias reparten las particiones para balancear carga y dar resiliencia.

Manejo de errores. Validación de payload, control de reintentos con backoff y derivación a un Dead Letter Topic ante fallos no recuperables. Los consumidores son idempotentes (RNF-03).

Microservicio administrador de Kafka. ms-admin-kafka permite crear, modificar y eliminar tópicos y configuraciones, e inspeccionar tópicos y grupos de consumidores.

Kafka UI y despliegue. Archivos docker-compose para el clúster de RabbitMQ (se mantiene), la arquitectura Kafka con tres Zookeeper y tres brokers, y Kafka UI para administrar y supervisar offsets, lag y distribución del consumo.

8.5 Continuidad de las etapas previas

Se verificará que la autenticación, el API Gateway, la validación de JWT, la persistencia y la topología de RabbitMQ con sus colas, DLQ y consumidores sigan operando. Kafka se incorpora de forma aditiva: el evento de cierre que consumen las colas de RabbitMQ mantiene su formato, de modo que los consumidores de la Etapa 2 no requieren cambios, aunque el productor de ese evento pase de ms-catalogo a ms-adjudicacion.

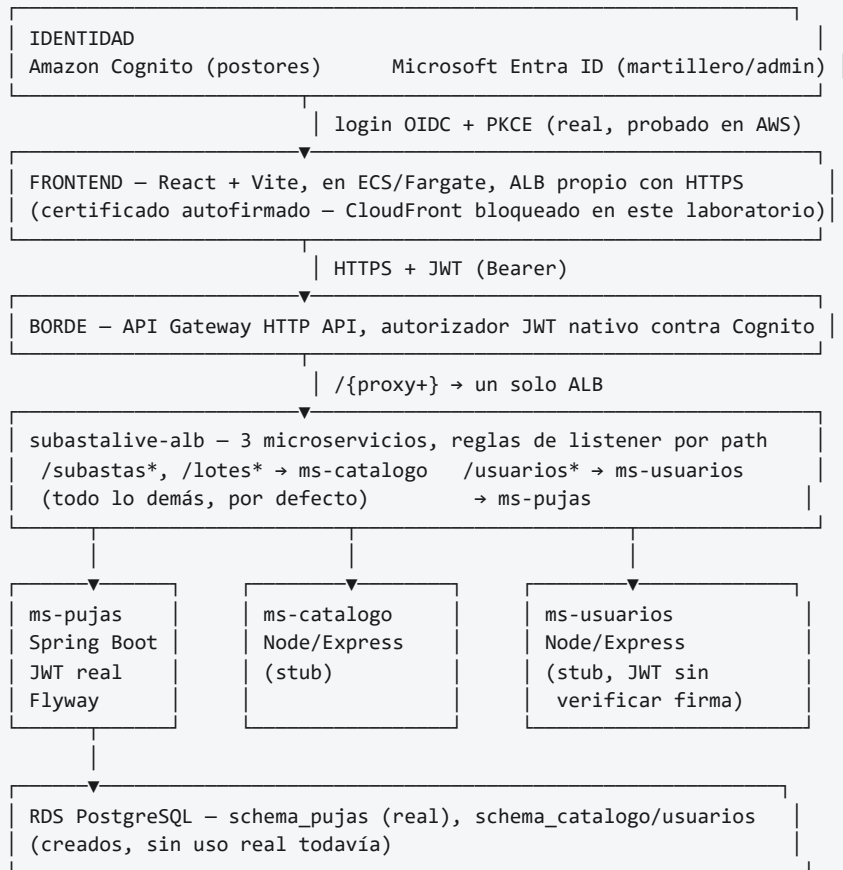
8.6 Verificación de requerimientos

Requerimiento	Elemento que lo cubre
R3.1	Configuración del cliente Kafka
R3.2	Productores y consumidores con DTO
R3.3	<code>ms-admin-kafka</code>
R3.4	Productores con serializers y <code>acks=all</code>
R3.5	Consumidores <code>@KafkaListener</code> con lógica separada
R3.6	Validación, reintentos y Dead Letter Topic
R3.7	Retención, cleanup policy, particiones y replicación
R3.8	Consumer groups de adjudicación y analítica
R3.9	<code>docker-compose</code> de RabbitMQ
R3.10	<code>docker-compose</code> de Kafka con Zookeeper y brokers
R3.11	<code>docker-compose</code> de Kafka UI
R3.12	Explicación de la arquitectura Kafka
R3.13	Justificación de tópicos, particiones y réplicas
R3.14	Demostración de Kafka UI
R3.15	Flujo de producción y consumo
R3.16	Diseño de los eventos
R3.17	Respuesta a eventos en tiempo real
R3.18	Administración de Kafka
R3.19	Supervisión de métricas
R3.20	Casos de uso de streaming
R3.21	Continuidad de las Etapas 1 y 2

9. Arquitectura final prevista

9.1 Arquitectura real de la Etapa 1

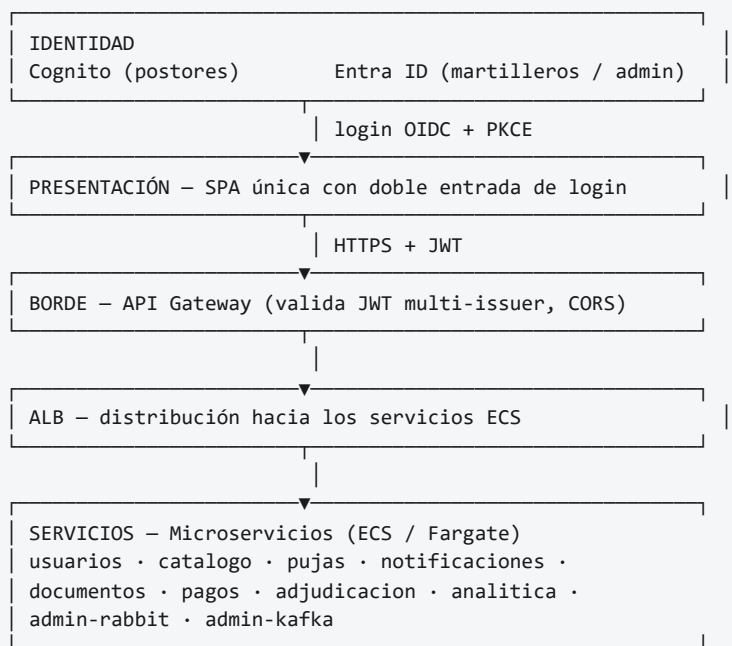
La arquitectura de la Etapa 1, ya construida y probada, queda así:

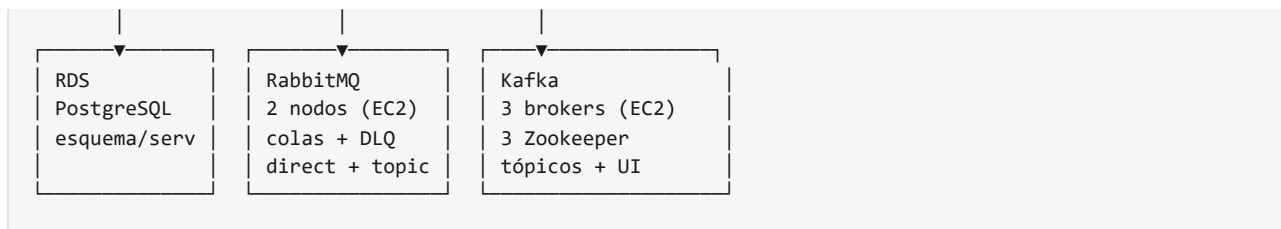


Todo lo anterior corre en subredes privadas salvo los 2 ALB (públicos); las tareas de ECS salen a internet solo a través del NAT Gateway, para descargar su imagen de ECR.

9.2 Arquitectura prevista al término de las tres etapas

Una vez completadas las Etapas 2 y 3, la arquitectura integrará:





La composición final se consolidará en la última entrega y podrá diferir de este esquema en función de los ajustes realizados durante el ramo. RDS mantiene el estado transaccional, RabbitMQ distribuye trabajos y Kafka distribuye y conserva eventos de dominio.

10. Trazabilidad con las pautas de evaluación

La verificación por requerimiento se encuentra en las secciones 6.4, 7.6 y 8.6. La tabla siguiente consolida la cobertura por etapa:

Etapas	Requerimientos del encargo	Requerimientos de la presentación	Sección
1	R1.1 a R1.8	R1.9 a R1.15	6.4
2	R2.1 a R2.11	R2.12 a R2.16	7.6
3	R3.1 a R3.11	R3.12 a R3.21	8.6

Elementos que las tres pautas exigen de forma transversal:

Elemento transversal	Etapas en que se establece
Backend como varios microservicios Java con Spring Boot	1
Frontend SPA modular con vistas funcionales	1
Integración con base de datos cloud	1
Validación de JWT en el backend	1
Login con IDaaS y uso del JWT	1
Archivos <code>.gitignore</code> por tecnología	1
Entrega mediante enlaces a GitHub	1, 2, 3

11. Consideraciones finales

Este plan traza la hoja de ruta de SubastaLive a lo largo de las tres evaluaciones de la asignatura. El enfoque incremental permitió llegar a una Entrega N°1 funcional que sirve de base real (no solo teórica) para las etapas siguientes, cumpliendo el criterio, presente en las pautas, de incorporar nuevas capacidades sin afectar lo ya construido.

Las historias de usuario y sus requisitos ordenan el desarrollo desde la necesidad de cada rol hasta el requerimiento técnico y la etapa que lo materializa, de modo que la funcionalidad queda trazada de extremo a extremo. Las decisiones de arquitectura —cómputo en ECS/Fargate, persistencia con esquema por servicio, dos plataformas de mensajería complementarias y consistencia mediante consumidores idempotentes— se adoptan priorizando la solución más simple que resulte robusta y defendible.

Algunas lecciones concretas que la Entrega N°1 dejó, no anticipadas en la planificación original:

- **Construir la infraestructura a mano primero, y solo después capturarla en Terraform, resultó más rápido de aprender que empezar directo con Infraestructura como Código** — cada decisión se entendió resolviendo el problema real en la consola, antes de automatizarla.
- **Los laboratorios de AWS Academy tienen restricciones reales que no se pueden anticipar solo leyendo documentación** (CloudFront bloqueado por completo, no solo Origin Access Control).
- **La rúbrica genérica de la asignatura usa terminología ("BFF") que no coincide con el patrón arquitectónico del mismo nombre** — vale la pena confirmar con el docente la interpretación exacta antes de construir algo que la pauta no pedía en realidad.
- **Coordinar el trabajo en equipo sobre el mismo repositorio requiere que todos manejen git de la misma forma** — una implementación real de `ms-usuarios` llegó en una rama sin historia compartida (clonada como ZIP en vez de `git clone`), lo que impidió un merge normal.

Las decisiones descritas para las Etapas 2 y 3 son orientaciones sujetas a ajuste. A medida que el ramo avance, el equipo refinará la topología de mensajería, el diseño de los eventos, la composición de los microservicios y los parámetros de configuración. Los elementos señalados como evolución (RDS Proxy, Transactional Outbox, mitigación de hot partitioning) se documentan como camino futuro y no forman parte del alcance de las entregas.

12. Referencias

- Apache Software Foundation. *Apache Kafka Documentation*. <https://kafka.apache.org/documentation/>
- Broadcom. *RabbitMQ Documentation*. <https://www.rabbitmq.com/docs>
- Spring. *Spring for Apache Kafka Reference Documentation*. <https://docs.spring.io/spring-kafka/reference/>
- Spring. *Spring AMQP Reference Documentation*. <https://docs.spring.io/spring-amqp/reference/>
- Spring. *Spring Security Reference*. <https://docs.spring.io/spring-security/reference/>
- Amazon Web Services. *Amazon ECS Developer Guide*. <https://docs.aws.amazon.com/ecs/>
- Amazon Web Services. *Amazon API Gateway Developer Guide*. <https://docs.aws.amazon.com/apigateway/>
- IETF. *RFC 7636 — Proof Key for Code Exchange by OAuth Public Clients*. <https://datatracker.ietf.org/doc/html/rfc7636>
- Este mismo repositorio: `despliegue-aws.md` — guía completa de despliegue en AWS, con cada error real y su solución.
- `infra-terraform/` — la misma infraestructura, capturada como código.
- `README.md` — contratos de API, convenciones compartidas y CI/CD.

13. Anexo — Credenciales de prueba para la demostración

Cuentas de prueba en un laboratorio temporal de AWS Academy/Azure, de bajo riesgo real — se dejan en claro para que cualquiera del equipo o el docente pueda probar el sistema directamente.

Postor (Amazon Cognito): usuario `postor.prueba2@example.com`, contraseña `Contra_12345`.

Martillero (Microsoft Entra ID): usuario `martillero.prueba@jusalass.onmicrosoft.com`, contraseña `Contra_123456`.

Administrador (Microsoft Entra ID): usuario `administrador.prueba@jusalass.onmicrosoft.com`, contraseña `Contra_123456`.

14. Anexo — Incidentes reales encontrados y su solución

Incidente	Causa	Solución
Error al descargar la imagen de <code>ms-pujas</code>	La subred privada perdió la asociación a la tabla de rutas del NAT Gateway al recrearse	Re-asociar la subred a <code>subastalive-private-rt</code>
Target Group de <code>ms-pujas</code> "Unhealthy — Request timed out"	Puerto del contenedor quedó en 80 (por defecto) en vez de 8083	Nueva revisión de la Task Definition con el puerto correcto
ECS revertía solo el despliegue en loop	Grace period insuficiente para el arranque de Spring Boot (~90s)	Grace period en 240s + umbral saludable del Target Group en 2
Frontend rechazado por Cognito (callback URL)	Cognito exige <code>https://</code> salvo <code>localhost</code>	Certificado autofirmado (OpenSSL + ACM); CloudFront está bloqueado en este laboratorio
El "Salir" no pedía credenciales de nuevo	El logout solo limpiaba la sesión local, sin cerrar la sesión de SSO en el IdP	<code>signoutRedirect()</code> real para Entra ID; endpoint <code>/logout</code> propietario de Cognito
<code>ms-usuarios</code> desplegado con IP pública	Error de configuración al crear el Service de ECS	Pendiente de corregir en la consola real; ya corregido en la plantilla de Terraform
Rama de un compañero con historia de git desconectada	Clonó el proyecto como ZIP en vez de <code>git clone</code>	Pendiente: debe re-clonar y rehacer su rama desde <code>main</code>